



Python Scripting — Complete Notes

Readable, batteries-included · Automation, tooling & glue

 Topic Python Use Scripting / Automation Style Readable Batteries Included

A reader-friendly, all-in-one reference for Python scripting.

Table of Contents

1. What is Python
2. Running Python & Virtual Envs
3. Variables & Data Types
4. Strings & f-strings
5. Data Structures
6. Control Flow & Comprehensions
7. Functions
8. Modules & Packages
9. File I/O & Context Managers
10. Error Handling
11. Classes & OOP
12. Scripting Toolkit (argparse, subprocess...)
13. Pythonic Idioms
14. Best Practices
15. Quick Mental Model
16. Common Interview Questions

1. 🐍 What is Python

Python is a **high-level, readable, general-purpose** language famous for clean syntax and a huge **standard library** ("batteries included"). For scripting it's the go-to when Bash gets awkward — real data structures, error handling, and libraries make it ideal for automation, tooling, data tasks, and APIs.

💡 **Mental model:** Python reads almost like pseudocode and trades a little speed for **developer productivity**. Use it when a script needs **logic, data structures, or maintainability**; use Bash for short command orchestration.

Common uses: automation scripts, CLI tools, data processing/ETL, web backends, DevOps tooling, testing, and ML/AI.

2. Running Python & Virtual Envs

```
python3 script.py      # run a script
python3 -c "print(1+1)" # one-liner
python3                # interactive REPL
```

```
#!/usr/bin/env python3  # shebang → chmod +x then ./script.py
print("hello")
```

Virtual environments isolate per-project dependencies:

```
python3 -m venv .venv      # create
source .venv/bin/activate  # activate (Linux/macOS)
pip install requests       # install into the venv
pip freeze > requirements.txt # pin deps
pip install -r requirements.txt
```

Always use a **venv** (or `uv` / `poetry`) per project — never `pip install` globally. `python -m venv` ships with Python.

3. Variables & Data Types

```
name = "Alice"      # str
age = 30             # int
price = 9.99         # float
active = True        # bool
nothing = None       # NoneType
```

- **Dynamically typed** (no type declared) but **strongly typed** (no implicit `"3" + 4`).
 - Everything is an **object**; check types with `type(x)` / `isinstance(x, int)` .
 - **Optional type hints** improve readability & tooling: `def add(a: int, b: int) -> int: .`
 - **Mutable** (list, dict, set) vs **immutable** (int, str, tuple, frozenset) — matters for function args and dict keys.
-

4. Strings & f-strings

```
name, n = "Bob", 3
s = f"Hello {name}, you have {n} messages" # f-string (preferred)
s.upper(); s.lower(); s.strip()
s.split(",") # → list
", ".join(["a", "b"]) # → "a,b"
s.replace("a", "b")
"py" in "python" # membership → True
s[0:2]; s[::-1] # slicing / reverse
```

f-strings (`f"...{expr}..."`) are the modern way to format — readable and fast. Strings are **immutable**; methods return new strings.

5. Data Structures

Type	Syntax	Traits
list	<code>[1, 2, 3]</code>	Ordered, mutable , duplicates OK
tuple	<code>(1, 2, 3)</code>	Ordered, immutable
dict	<code>{"k": "v"}</code>	Key→value, ordered (3.7+), fast lookup
set	<code>{1, 2, 3}</code>	Unordered, unique elements

```
nums = [3, 1, 2]
nums.append(4); nums.sort(); nums[0] # list ops
person = {"name": "Al", "age": 30}
person["role"] = "dev" # add key
person.get("missing", "default") # safe lookup
for k, v in person.items(): ...
unique = set([1, 1, 2]) # {1, 2}
```

Pick by need: **list** = ordered collection, **dict** = lookups by key, **set** = uniqueness/membership, **tuple** = fixed record.

6. Control Flow & Comprehensions

```
if age >= 18:
    print("adult")
elif age > 0:
    print("minor")
else:
    print("invalid")

for i in range(5): ...           # 0..4
for item in mylist: ...
while condition: ...
# break / continue / else-on-loop

# Comprehensions – concise transforms
squares = [x*x for x in range(10)]
evens   = [x for x in nums if x % 2 == 0]
lookup  = {k: len(k) for k in words}
```

Indentation defines blocks (no braces) — consistent whitespace is syntax. **Comprehensions** are the Pythonic way to build lists/dicts/sets from loops.

7. Functions

```
def greet(name, greeting="Hello"):    # default arg
    return f"{greeting}, {name}"

def total(*args, **kwargs):           # variadic
    # args = tuple of positional, kwargs = dict of keyword
    return sum(args)

square = lambda x: x * x              # anonymous (small)
greet("Al"); greet("Al", greeting="Hi")
```

- **Default arguments** make params optional. ***args** captures extra positionals (tuple); ****kwargs** captures extra keyword args (dict).
-  **Never use a mutable default** (`def f(x=[])`) — it's shared across calls; use `None` and create inside.
- Functions are **first-class** (pass/return them); **lambda** is for tiny inline functions.

8. 📦 Modules & Packages

```
import math
from datetime import datetime
import json as j
from mymodule import helper

print(math.sqrt(16))
```

- A **module** = a `.py` file; a **package** = a directory of modules.
- `if __name__ == "__main__":` — code under it runs only when the file is executed directly, not when imported (the standard script entry-point guard).

```
def main():
    ...

if __name__ == "__main__":
    main()
```

9. 📄 File I/O & Context Managers

```
with open("data.txt") as f:      # auto-closes (context manager)
    for line in f:
        print(line.rstrip())

with open("out.txt", "w") as f:  # "w" write, "a" append, "r" read
    f.write("hello\n")

import json
with open("config.json") as f:
    cfg = json.load(f)          # parse JSON → dict
```

Always use `with open(...)` — the context manager closes the file even on error. Modes: `r` read, `w` overwrite, `a` append, `b` binary.

10. 🛡️ Error Handling

```
try:
    result = 10 / x
except ZeroDivisionError:
    print("can't divide by zero")
except (ValueError, TypeError) as e:
    print(f"bad input: {e}")
else:
    print("ok:", result)      # runs if no exception
finally:
    print("always runs")     # cleanup

if x < 0:
    raise ValueError("x must be non-negative")  # raise your own
```

- Python uses **exceptions** (EAFP — "easier to ask forgiveness than permission"): try the action, handle the failure.
- Catch **specific** exceptions, not bare `except:`. `finally` always runs (cleanup); `else` runs only if no exception.

11. 🏠 Classes & OOP

```
class Dog:
    species = "Canis"          # class attribute (shared)

    def __init__(self, name):   # constructor
        self.name = name       # instance attribute

    def bark(self):             # method (self = instance)
        return f"{self.name} says woof"

class Puppy(Dog):              # inheritance
    def bark(self):
        return super().bark() + "!" # override + call parent

d = Puppy("Rex")
d.bark()                       # "Rex says woof!"
```

- `__init__` initializes a new instance; `self` is the instance. Class attrs are shared; instance attrs are per-object.
- Supports **inheritance**, `super()`, and **dunder methods** (`__str__`, `__repr__`, `__eq__`, `__len__`). `@dataclass` auto-generates boilerplate for data-holding classes.

12. 🧰 Scripting Toolkit (argparse, subprocess...)

The stdlib modules that make Python great for scripts:

```

import argparse, subprocess, os, sys
from pathlib import Path

# CLI arguments
p = argparse.ArgumentParser()
p.add_argument("path"); p.add_argument("--verbose", action="store_true")
args = p.parse_args()

# Run shell commands
out = subprocess.run(["ls", "-la"], capture_output=True, text=True, check=True)
print(out.stdout)

# Paths & files (modern)
for f in Path(".").glob("*.log"):
    print(f.name, f.stat().st_size)

os.environ.get("HOME")          # env vars
sys.argv; sys.exit(1)          # raw args / exit code

```

Module	Use
<code>argparse</code>	Parse CLI flags/args
<code>subprocess</code>	Run external commands
<code>pathlib</code>	Modern path/file handling
<code>os</code> / <code>sys</code>	Env, args, exit codes
<code>json</code> / <code>csv</code>	Data formats
<code>logging</code>	Proper logging (over <code>print</code>)
<code>requests</code>	HTTP (third-party, ubiquitous)

13. ✨ Pythonic Idioms

```

a, b = b, a          # swap (tuple unpacking)
for i, x in enumerate(items): ... # index + value
for k, v in d.items(): ...      # dict iteration
for a, b in zip(list1, list2): ... # parallel iteration
text = " ".join(words)         # build strings
val = d.get(key, default)       # safe dict access
result = x if cond else y       # ternary
squares = [x*x for x in nums]   # comprehension over loop
with open(f) as fh: ...        # context managers for resources

```

"Pythonic" = readable, idiomatic code. Prefer comprehensions, unpacking, `enumerate` / `zip`, `with`, and the standard library over manual/verbose patterns. Follow **PEP 8** (style guide).

14. Best Practices

- Use a **virtual environment** per project; pin deps in `requirements.txt` (or `pyproject.toml`).
 - Follow **PEP 8**; format with **black**, lint with **ruff/flake8**, type-check with **mypy**.
 - Add **type hints** for clarity and tooling.
 - Guard the entry point with `if __name__ == "__main__":`.
 - Catch **specific** exceptions; use `with` for files/resources; use **logging** not `print` in real tools.
 - Prefer **comprehensions**, **pathlib**, **f-strings**; avoid mutable default arguments.
 - Write **functions small and testable**; add **docstrings**; test with **pytest**.
 - Choose Python over Bash once a script needs real logic, data structures, or error handling.
-

15. Quick Mental Model

- **Python = readable, batteries-included scripting** — reach for it when Bash gets awkward.
 - **Dynamically but strongly typed**; everything's an object; **indentation = blocks**.
 - Core structures: **list** (ordered), **dict** (key→value), **set** (unique), **tuple** (immutable).
 - **f-strings** for formatting; **comprehensions** for transforms; `with` for resources.
 - Functions: defaults, `*args` / `**kwargs`, first-class; avoid **mutable default args**.
 - Handle errors with **try/except/finally** (EAFP); catch specific exceptions.
 - Scripting toolkit: **argparse** (CLI), **subprocess** (commands), **pathlib** (files), **logging**.
 - **venv per project**, **PEP 8**, `if __name__ == "__main__":`, test with **pytest**.
-

16. Common Interview Questions

Rapid-fire questions interviewers ask about Python:

- **Q: List vs tuple?** — List is mutable and ordered; tuple is immutable (and can be a dict key / set element).
- **Q: Is Python statically or dynamically typed?** — Dynamically typed (no declared types) but strongly typed (no implicit type coercion like `"3" + 4`).
- **Q: What does `if __name__ == "__main__":` do?** — Runs that block only when the file is executed directly, not when imported as a module.
- **Q: `*args` vs `**kwargs`?** — `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict.
- **Q: What's the mutable-default-argument gotcha?** — A default like `def f(x=[])` is created once and shared across calls; use `None` and create the value inside.
- **Q: What is a list comprehension?** — A concise expression to build a list from an iterable: `[x*x for x in nums if x>0]`.
- **Q: What is a context manager / `with`?** — An object that sets up and tears down a resource automatically (e.g. closes a file) even if an error occurs.
- **Q: How do you handle errors?** — `try/except` specific exceptions, optional `else` (no error) and `finally` (always runs); raise with `raise`.
- **Q: What is a virtual environment and why?** — An isolated per-project Python + dependency set, so projects don't conflict and you don't pollute the system Python.

- **Q: Shallow vs deep copy?** — Shallow copies the outer object but shares nested objects; `copy.deepcopy` recursively copies everything.
 - **Q: How do you run a shell command from Python?** — `subprocess.run(..., capture_output=True, text=True, check=True)` .
 - **Q: Bash vs Python for scripting?** — Bash for short command orchestration/glue; Python once you need data structures, real logic, error handling, or maintainability.
-

 Notes compiled as a quick reference for Python scripting.